

Complejidad Computacional

Guía práctica N°1 – @valnrms

Fecha de compilación: 13/04/2025.

Ejercicio 1.

Para los siguientes pares de funciones $f, g : \mathbb{N} \rightarrow \mathbb{R}$ decidir si $g = o(f)$, $f = o(g)$ o $f = \Theta(g)$.¹

¡Por si no sabías! Se dice que $f = o(g) \iff \forall c \in \mathbb{R}_{>0}, \exists n_0 \in \mathbb{N}_{>0} : \forall n \geq n_0, f(n) \leq c \cdot g(n)$.

Es similar pero distinto a la notación $f = O(g) \iff \exists c \in \mathbb{R}_{>0}, \exists n_0 \in \mathbb{N}_{>0} : \forall n \geq n_0, f(n) \leq c \cdot g(n)$.

A su vez, $f = \Theta(g) \iff \exists c_1, c_2 \in \mathbb{R}_{>0}, \exists n_0 \in \mathbb{N}_{>0} : \forall n \geq n_0, c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$.

1.a $f(n) = n^2, g(n) = 2n^2 + 100\sqrt{n}$.

Propiedad del límite para el cálculo de complejidades:

Sean $f, g : \mathbb{N} \rightarrow \mathbb{R}_{>0}$.

Si existe: $\lim_{n \rightarrow +\infty} \frac{f(n)}{g(n)} = \ell \in \mathbb{R}_{>0} \cup \{+\infty\}$, entonces:

- $f \in \Theta(g) \iff 0 < \ell < +\infty$
- $f \in O(g)$ y $f \notin \Omega(g) \iff \ell = 0$
- $f \in \Omega(g)$ y $f \notin O(g) \iff \ell = +\infty$

Propongo que $f = \Theta(g)$, probémoslo.

$$n^2 = \Theta(2n^2 + 100\sqrt{n})$$

$\iff$

propiedad del límite para el cálculo de complejidad

$$\lim_{n \rightarrow +\infty} \frac{n^2}{2n^2 + 100\sqrt{n}} = \lim_{n \rightarrow +\infty} \frac{n^2}{2n^2 \left(1 + \frac{100\sqrt{n}}{2n^2}\right)} = \lim_{n \rightarrow +\infty} \frac{1}{2 \left(1 + \frac{100\sqrt{n}}{2n^2}\right)} = \frac{1}{2}$$

1.b $f(n) = n^{100}, g(n) = 2^{\frac{n}{100}}$.

Se puede extender la propiedad del límite a Little-o y Little- ω :

Si $g(n)$ es distinto de cero, o al menos se vuelve distinto de cero más allá de cierto punto n_0 ,

- la relación $f = o(g)$ es equivalente a $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$
- la relación $f = \omega(g)$ es equivalente a $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$

Propongo que $f = o(g)$, probémoslo.

$$n^{100} = o\left(2^{\frac{n}{100}}\right)$$

$\iff$

propiedad del límite extendida para el cálculo de complejidad

$$\lim_{n \rightarrow \infty} \frac{n^{100}}{2^{\frac{n}{100}}} = \lim_{n \rightarrow \infty} e^{\ln\left(\frac{n^{100}}{2^{\frac{n}{100}}}\right)} = \lim_{n \rightarrow \infty} e^{\ln(n^{100}) - \ln\left(2^{\frac{n}{100}}\right)} = \lim_{n \rightarrow \infty} e^{100 \ln(n) - \frac{n}{100} \ln(2)} = \lim_{n \rightarrow \infty} e^{100 \ln(n) - \frac{n}{100} \ln(2)} \xrightarrow[n \rightarrow \infty]{} -\infty = 0$$

¹En general podría no pasar ninguna de las 3, ver Ejercicio 5.

$$1.c \ f(n) = n^{100}, g(n) = 2^{n^{\frac{1}{100}}}$$

$$1.d \ f(n) = \sqrt{n}, g(n) = 2^{\sqrt{\log n}}$$

$$1.e \ f(n) = n^{20}, g(n) = 2^{\log^2 n}$$

$$1.f \ f(n) = 50n, g(n) = n \log n$$

Ejercicio 2.

Indicar, para cada una de las siguiente funciones definidas recursivamente, su orden de crecimiento. Es decir, para cada función f encontrar una función g definida de forma cerrada (i.e. no recursivamente) tal que $f = \Theta(g)$. De ser posible, encontrar una expresión cerrada para f .²

$$2.a \ f(n) = f(n-1) + 10$$

Busco expresión cerrada para f .

- $f(0) = f(1) = \dots = f(10) = 1$
- $f(11) = f(10) + 10 = 1 + 10 = 11$
- $f(12) = f(11) + 10 = 11 + 10 = 21$
- $f(13) = f(12) + 10 = 21 + 10 = 31$
- $f(14) = f(13) + 10 = 31 + 10 = 41$
- ...

Propongo $f(n) = \begin{cases} 10n-99 & \text{si } n > 10 \\ 1 & \text{sino} \end{cases}$. Luego, $f = \Theta(n)$.

$$2.b \ f(n) = f(n-1) + n$$

Busco expresión cerrada para f .

- $f(0) = f(1) = \dots = f(10) = 1$
- $f(11) = f(10) + 11 = 1 + 11 = 12$
- $f(12) = f(11) + 12 = 12 + 12 = 24$
- $f(13) = f(12) + 13 = 24 + 13 = 37$
- $f(14) = f(13) + 14 = 37 + 14 = 51$
- ...

Propongo $f(n) = \begin{cases} 1 + \sum_{i=11}^n i & \text{si } n > 10 \\ 1 & \text{sino} \end{cases} = \begin{cases} \frac{n^2+n}{2} - 54 & \text{si } n > 10 \\ 1 & \text{sino} \end{cases}$. Luego, $f = \Theta(n^2)$.

$$2.c \ f(n) = 2f(n-1)$$

Busco expresión cerrada para f .

- $f(0) = f(1) = \dots = f(10) = 1 = 2^0$
- $f(11) = 2f(10) = 2 * 1 = 2 = 2^1$
- $f(12) = 2f(11) = 2 * 2 = 4 = 2^2$
- $f(13) = 2f(12) = 2 * 2^2 = 8 = 2^3$
- $f(14) = 2f(13) = 2 * 2^3 = 16 = 2^4$
- ...

Propongo $f(n) = \begin{cases} 2^{n-10} & \text{si } n > 10 \\ 1 & \text{sino} \end{cases}$. Luego, $f = \Theta(2^n)$.

¡¿Te acordás?! El *teorema maestro* es una herramienta para resolver **relaciones recursivas** de la forma: $T(n) = aT(\frac{n}{b}) + f(n)$, donde $a \geq 1$ y $b > 1$ son constantes y f es el costo del trabajo hecho fuera de las llamadas recursivas, que incluye el costo de la división del problema y el costo de unir las soluciones de los subproblemas.

Podemos separar al **teorema maestro** en 3 casos:

²Para todos los casos, asumir que $f(i) = 1$ para todo $i \leq 10$.

- Si $f(n) = O(n^c)$, donde $c < \log_b a \Rightarrow T(n) = \Theta(n^{\log_b a})$
- Si $f(n) = \Theta(n^{\log_b a}) \Rightarrow T(n) = \Theta(n^{\log_b a} \log(n))$
- Si $f(n) = \Omega(n^c)$, donde $c > \log_b a \wedge af(\frac{n}{b}) \leq kf(n)$ para algún $k < 1$ y un n suficientemente grande $\Rightarrow T(n) = \Theta(f(n))$

2.d $f(n) = 2f(\frac{n}{2}) + 10$

Utilizando el *teorema maestro*:

- $a = 2$
- $b = 2$
- $f(n) = 10 = O(n^0) \Rightarrow c = 0$

Luego, $0 < \log_b a = \log_2 2 = 1$, por lo que **estamos en el caso 1**.

Por lo tanto, $f(n) = \Theta(n^{\log_b a}) = \Theta(n^1) = \Theta(n)$.

2.e $f(n) = 2f(\frac{n}{2}) + n$

Utilizando el *teorema maestro*:

- $a = 2$
- $b = 2$
- $f(n) = n = \Theta(n^1) \Rightarrow c = 1$

Luego, $1 = \log_2 2$, por lo que **estamos en el caso 2**.

Por lo tanto, $f(n) = \Theta(n^{\log_b a} \log(n)) = \Theta(n^1 \log(n)) = \Theta(n \log(n))$.

2.f $f(n) = 2f(\frac{n}{2}) + n^3$

Utilizando el *teorema maestro*:

- $a = 2$
- $b = 2$
- $f(n) = n^3$.

$\log_2 2 = 1$, veamos los distintos casos:

- ▶ Caso 1: $n^3 = O(n^c)$, donde $c < 1$.
Falso, no existe tal c .
- ▶ Caso 2: $n^3 = \Theta(n^1)$.
Falso, n^3 crece más rápido que n^1 .
- ▶ Caso 3: $n^3 = \Omega(n^c)$, donde $c > 1$. Verdadero para $c = 3$. Veamos si se cumple la otra parte de la hipótesis.
¿Es cierto que $2\frac{n^3}{2} \leq kn^3$ para algún k y n suficientemente grande?, sí, con $k = 1$ y cualquier n .

Luego, **estamos en el caso 3**.

Por lo tanto, $f(n) = \Theta(f(n)) = \Theta(n^3)$.

Para el resto de los ejercicios, sea $F = \{f : \mathbb{N} \rightarrow \mathbb{R} \mid f \text{ es no decreciente}\}$ el conjunto de funciones no decrecientes. Definimos la relación $\sim_{\Theta}$ sobre F como $f \sim_{\Theta} g \iff f = \Theta(g)$.
Equivalentemente definimos $\sim_o, \sim_{\omega}, \sim_{\Omega}$.

Ejercicio 3.

Probar que $\sim_{\Theta}$ es una relación de equivalencia, y que tiene infinitas clases de equivalencia.³

Para probar que $\sim_{\Theta}$ es una relación de equivalencia, debemos probar que cumple las 3 propiedades:

- Reflexividad: $\forall f \in F : f \sim_{\Theta} f$.
 - ▶ $f = \Theta(f) \iff \exists c_1, c_2 \in \mathbb{R}_{>0}, \forall n \geq n_0 \in \mathbb{R}_{>0} : c_1 \cdot f(n) \leq f(n) \leq c_2 \cdot f(n)$
 - ▶ Con $c_1 = c_2 = 1$ y $n_0 = 0$ se cumple la relación.

³Opcionalmente, probar que la cantidad de clases de equivalencia es no numerable.

- Simetría: $\forall f, g \in F : f \sim_{\Theta} g \Rightarrow g \sim_{\Theta} f$.
 - $f \sim_{\Theta} g \Leftrightarrow f = \Theta(g)$

$$\Leftrightarrow \exists c_1, c_2 \in \mathbb{R}_{>0}, \forall n \geq n_0 \in \mathbb{R}_{>0} : c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$

$$\Leftrightarrow c_1 \cdot g(n) \leq f(n) \wedge c_2 \cdot g(n) \geq f(n)$$

$$\Leftrightarrow g(n) \leq \frac{1}{c_1} f(n) \wedge g(n) \geq \frac{1}{c_2} f(n)$$

$$\Leftrightarrow \frac{1}{c_2} f(n) \leq g(n) \wedge \frac{1}{c_1} f(n) \geq g(n)$$

$$\Leftrightarrow \frac{1}{c_2} f(n) \leq g(n) \leq \frac{1}{c_1} f(n)$$

$$\Leftrightarrow g = \Theta(f)$$
- Transitividad: $\forall f, g, h \in F : f \sim_{\Theta} g \wedge g \sim_{\Theta} h \Rightarrow f \sim_{\Theta} h$.
 - $f \sim_{\Theta} g \Leftrightarrow f = \Theta(g) \Leftrightarrow c_1 g \leq f \leq c_2 g$
 - $g \sim_{\Theta} h \Leftrightarrow g = \Theta(h) \Leftrightarrow c_3 h \leq g \leq c_4 h$
 - Quiero ver que $h = \Theta(f)$, o sea, que $c_5 h \leq f \leq c_6 h$
 - Asumiendo verdadera la hipótesis,
 - $f \leq c_2 g \leq c_2 c_4 h \Rightarrow c_6 = c_2 c_4$
 - $c_1 c_3 h \leq c_1 g \leq f \Rightarrow c_5 = c_1 c_3$
 - Por lo tanto, $f = \Theta(h)$.

Luego, $\sim_{\Theta}$ es una relación de equivalencia.

Ejercicio 4. ★

Decidir y demostrar para qué casos $\sim_{\star}$ es reflexiva (con $\star \in \{o, O, \omega, \Omega\}$). Hacer lo mismo para simetría y transitividad.

Lo debería hacer, pero tengo poco tiempo. Me parece un ejercicio importante, por eso la ★.

Ejercicio 5.

Probar que existen funciones f y g tales que $f \neq O(g)$ y $g \neq O(f)$.

Consultar.

Ejercicio 6.

Probar que para todo $k > 0$ y $c > 1$ vale que $n^k = o(c^n)$. Concluir que todo polinomio crece más lento que cualquier función exponencial.

$$n^k = o(c^n) \Leftrightarrow \lim_{n \rightarrow \infty} \frac{n^k}{c^n} = 0$$

Resolvamos el límite y veamos si se cumple la igualdad.

$$\lim_{n \rightarrow \infty} \frac{n^k}{c^n} = \lim_{n \rightarrow \infty} e^{\ln\left(\frac{n^k}{c^n}\right)} = \lim_{n \rightarrow \infty} e^{\ln(n^k) - \ln(c^n)} = \lim_{n \rightarrow \infty} e^{k \ln(n) - n \ln(c)} \stackrel{k \ln(n) - n \ln(c) \xrightarrow{n \rightarrow \infty} -\infty}{=} 0$$

Ejercicio 7.

Sea $Poly_{\leq} \subseteq F$ el conjunto de funciones acotadas por arriba por un polinomio (i.e. $Poly_{\leq} = \{f \in F : \exists k \geq 0 \text{ tal que } f \in O(n^k)\}$), y $Exp_{\geq}$ el conjunto de funciones acotadas por debajo por alguna función exponencial (i.e. $Exp_{\geq} = \{f \in F : \exists c > 1 \text{ tal que } f \in \Omega(c^n)\}$). Probar que $F \neq Poly_{\leq} \cup Exp_{\geq}$. Es decir, probar que existen funciones que crecen más rápido que cualquier polinomio pero más lento que cualquier función exponencial. **Ayuda:** considerar $f(n) = n^{\log n}$.

Debería probar que $n^{\log n} \in O(c^n)$ y $n^{\log n} \in \Omega(n^k)$ para todo $k \geq 0$ y $c > 1$.

Veamos primero que $n^{\log n}$ crece más lento que cualquier función exponencial. Esto ocurre si y solamente si:

$$n^{\log n} \in O(c^n) \iff \lim_{n \rightarrow +\infty} \frac{n^{\log n}}{c^n} = 0$$

Calculemos el límite y veamos si se cumple la igualdad.

$$\lim_{n \rightarrow +\infty} \frac{n^{\log n}}{c^n} = \lim_{n \rightarrow +\infty} e^{\ln\left(\frac{n^{\log n}}{c^n}\right)} = \lim_{n \rightarrow +\infty} e^{\log n \ln n - n \ln c} = \lim_{n \rightarrow +\infty} e^{\frac{(\ln n)^2}{\ln 10} - n \ln c} = 0$$

Veamos ahora que $n^{\log n}$ crece más rápido que cualquier polinomio. Esto ocurre si y solamente si:

$$n^{\log n} \in \Omega(n^k) \iff \lim_{n \rightarrow +\infty} \frac{n^{\log n}}{n^k} = +\infty$$

Calculemos el límite y veamos si se cumple la igualdad.

$$\begin{aligned} \lim_{n \rightarrow +\infty} \frac{n^{\log n}}{n^k} &= \lim_{n \rightarrow +\infty} e^{\ln\left(\frac{n^{\log n}}{n^k}\right)} = \lim_{n \rightarrow +\infty} e^{\log n \ln n - k \ln n} = \\ \lim_{n \rightarrow +\infty} e^{(\log n - k) \ln n} &= \lim_{n \rightarrow +\infty} e^{\ln n (\log n - k)} = \lim_{n \rightarrow +\infty} e^{(\log n - k) \ln n} = +\infty \end{aligned}$$

Ejercicio 8.

Decidir si son ciertas las siguientes afirmaciones:

8.a $f(n) = \Theta(g(n))$ entonces $2^{f(n)} = \Theta(2^{g(n)})$

Sí porque es todo creciente.

8.b $f(n) = 2^{\Theta(\log n)}$ si y solamente si $f(n)$ es un polinomio.⁴

Quiero ver que,

$$c_1 2^{\log n} \leq f(n) \leq c_2 2^{\log n} \iff f(n) \in \Theta(n^k) \text{ para algún } k \geq 0$$

Consultar.

Ejercicio 9.

Argumentar que los siguientes modelos de cómputo son polinomialmente equivalentes a una máquina de Turing de una sola cinta de trabajo infinita en un solo sentido. Estimar la complejidad de la simulación.

9.a

Máquina de Turing con una sola cinta de trabajo infinita en ambos sentidos.

Partiendo de la posición 0 (del start), se puede “plegar” la cinta bi-infinita a solo un lado, y simular la máquina de Turing de una sola cinta infinita en un solo sentido.

Usamos un alfabeto $\{\triangleright\} \cup \{0, 1, \square\}^2$. De manera tal que el ab en un celda, con $a, b \in \{0, 1, \square\}$, representa el símbolo a en la posición i de la cinta, y el símbolo b en la posición $-i$ de la cinta.

Está todo a misma distancia del 0. Podemos pensar que cada vez que quieras pasar de negativos a positivos volvés al 0 y de ahí te movés, lo que hace que el cómputo sea exactamente el mismo. Es decir, usamos los estados para diferenciar entre parte negativa y parte positiva de la cinta.

9.b

Máquina de Turing con k cintas de trabajo, todas infinitas en ambas direcciones, con $k \geq 1$.

⁴El conjunto $2^{\Theta(g(n))}$ es el conjunto de funciones $h \in F$ tales que existen constantes c_0 y c_1 con $2^{c_0 g(n)} \leq h(n) \leq 2^{c_1 g(n)}$ para todo n a partir de un cierto n_0 . Notar que podríamos extender esta idea y definir la clase de funciones $f(\Theta(g(n)))$ o equivalentemente para o, O, ω o Ω .

Se puede pensar en intercalar las posiciones, como en el apunte de Santi, de manera tal que tenés $k \cdot T(n) + 2$ configuraciones, que sigue siendo polinomial.

9.c

Máquina de Turing con acceso RAM (i.e. una máquina de Turing con una sola cinta de trabajo infinita en un solo sentido, que tiene una cinta especial adicional sobre la cual puede escribir, y tiene una instrucción especial que mueve el puntero de la cinta de trabajo a la dirección que está escrita en la cinta especial)⁵.

9.d

Máquina de Turing con memoria bidimensional infinita (i.e. reemplaza la cinta infinita unidireccional por una matriz M con casilleros en el rango $[0, \infty) \times [0, \infty)$).

Ejercicio 10.

Probar por cardinalidad que hay lenguajes que no son computables. Definir uno por diagonalización.

Ejercicio 11.

Argumentar que toda máquina de Turing que use tiempo $T(n)$ puede ser simulada por otra máquina que sea *oblivious* y use tiempo $T(n)^2$.

Ejercicio 12.

Argumentar que toda máquina de Turing que use espacio $T(n)$ puede ser simulada por otra máquina que sea *oblivious* y use espacio $T(n)$.

Ejercicio 13.

Argumentar que toda máquina de Turing con alfabeto Γ con $|\Gamma| \geq 2$ que use tiempo $T(n)$ puede ser simulada por una máquina de Turing que use tiempo $T(n) \log |\Gamma|$.

⁵Por simplicidad, asumimos que a través del acceso RAM el puntero solo puede moverse a posiciones de memoria ya visitadas. Caso contrario, la máquina termina con un Segmentation Fault.